A Reinforcement Learning Architecture for Spatio-Temporal Reallocation in Smart Container Terminals: Independent Proposal and Acceptance Policies with Counterfactual Cost Learning

Author Name[1]

Affiliation, Address email@example.com

Abstract. Container terminals assign each external-truck job to a yard block and an appointment time before the truck arrives. A fixed assignment, however, cannot reflect congestion that emerges later. This paper proposes a reinforcement learning architecture that jointly revisits the block and arrival time of an announced job during operation. Two independent policies decide whether to propose a reallocation and whether the destination block or time slot should accept it. A central procedure then commits only mutually accepted candidates that satisfy resource constraints. Moving one job affects not only that job but also the waiting time of later trucks, the crane work sequence, and vessel supply. To capture these delayed effects, each policy learns from cost differences between discrete-event simulation branches that start from the same state and differ in one action. Online decisions use only the trained networks. We evaluate the architecture in a continuous synthetic environment whose demand scale and time-of-day variation are based on ranges reported in the literature. Compared with no reallocation, the proposed architecture reduced total operating cost by 14.7%. It also cost less than a rule-based policy with the same spatio-temporal action range on 20 of 28 days ($p = 0.036$). Arrival-time adjustments accounted for 91.3% of the reduction, and the four most congested days accounted for 90.5%. These results suggest that spatio-temporal reallocation is most useful as a selective response to observed congestion rather than as a continuous intervention.

Keywords: Smart port · Container terminal · Truck arrival management · Yard block reallocation · Counterfactual learning · Reinforcement learning

1 Introduction

Container throughput continues to grow, but terminal land and equipment cannot expand at the same rate. The resulting pressure appears as congestion in yard and gate operations. Smart-port research therefore addresses not only automated equipment but also operational decisions. Studies of port efficiency show that automation, intelligence, and environmental design affect performance through

different channels. Actual gains thus depend on how a terminal turns observed information into operating decisions [1]. Existing work follows two main directions. On the temporal side, truck appointment systems spread arrival peaks and allow terminals and carriers to coordinate arrival times while retaining their own costs [2, 3]. On the spatial side, yard-operations research improves container placement, work sequencing, and crane-interference management [4–7]. Recent studies have also applied reinforcement learning to real-time yard-crane dispatching [8].

Most of these approaches assume that the working block and appointment time have already been fixed before the truck arrives. They improve operations within that assignment. During operation, however, demand may concentrate in particular time slots or blocks, while crane queues also change. An assignment that was reasonable when announced may then become inefficient. This paper revisits the assignment itself. For an announced job, it jointly decides *where* the job should be handled and *when* the truck should arrive. Moving one job affects later truck queues, the crane work sequence, and vessel supply. An immediate movement cost cannot capture these effects. Difference rewards and multi-agent counterfactual baselines address this problem by comparing one action with an alternative [9, 10]. The destination must also decide whether it can absorb the job, separately from any benefit to the proposing side. We therefore use independent proposal and acceptance policies, while a central procedure checks mutual consent and feasibility.

This study asks four questions. Can the two independent policies support spatio-temporal reallocation? Can expensive counterfactual learning remain separate from online inference? Does the architecture reduce operating cost? Finally, can the effects of the action range and of learning be distinguished? Our contributions are an *independent policy structure*, a *spatio-temporal action range*, *counterfactual cost learning*, a *separation of learning and operation*, and a *decomposable evaluation* of execution, action range, and learning effect. Sect. 2 reviews the relevant literature. Sect. 3 presents the architecture and learning signal, Sect. 4 describes the experimental environment, and Sect. 5 reports the results.

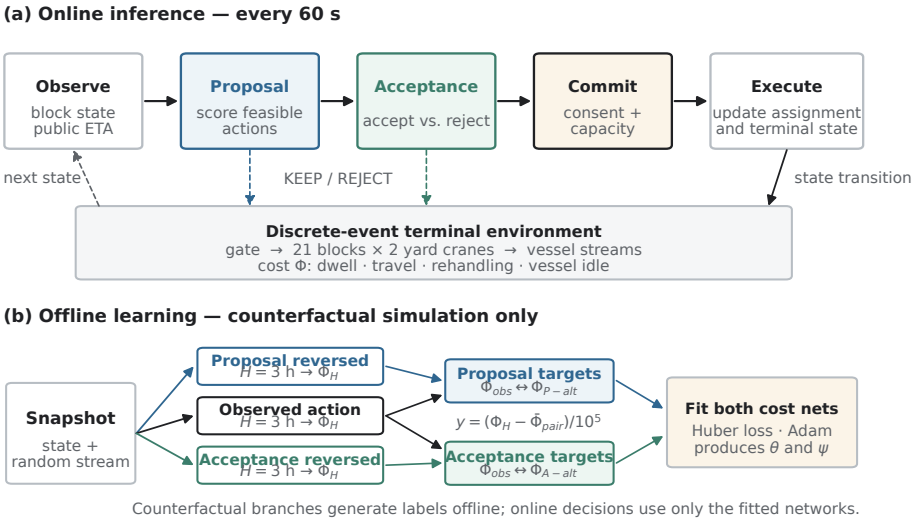

Fig. 1: Architecture and information flow. (a) Every 60 s the fitted proposal and acceptance networks score candidates, after which the central procedure commits only mutually accepted and feasible changes. Executing a decision advances the discrete-event terminal environment and produces the next observed state. (b) Offline learning clones the state and random stream immediately before a sampled decision, compares up to three worlds over a 3-h horizon, and fits the two networks to centred cost differences. The counterfactual branches are not invoked during online inference.

2 Related Work

2.1 Smart Ports and Terminal Operations

Yen et al. measure port efficiency with data envelopment analysis and then use Tobit regression to relate the scores to smart-port design attributes. This separates *how efficient a port is* from *what explains the difference*. Their results also treat automation, intelligence, and environmental design as distinct channels [1]. This distinction motivates attention to the decision layer in addition to the equipment layer.

On the temporal side, Chen et al. manage truck arrivals with time windows to relieve gate congestion. Their method reshapes the arrival profile instead of adding capacity [2]. Phan and Kim model a decentralised setting in which trucking companies and the terminal coordinate appointments while retaining their own costs [3]. This setting motivates our decision to model acceptance separately rather than absorb it into one objective. Kourounioti and Polydoropoulou use aggregate terminal data to model truck arrivals by time of day and show that arrivals are far from uniform [11]. The value of shifting an arrival therefore depends on both its original and target positions on the arrival curve. The environment in Sect. 4 reflects this time-of-day variation.

On the spatial side, Ng and Mak study yard crane scheduling in container terminals and sequence crane jobs to reduce truck waiting [4]. Speer and Fischer extend this problem to automated systems with several cranes that can interfere within the same block [5]. Petering and Murty use long-run simulation to study how block length and crane deployment affect terminal-wide performance [6]. We adopt their continuous, multi-week design with state carried forward. Schwientek et al. compare dispatching methods in simulation and show that their ranking depends on terminal conditions [7]. We therefore hold the dispatching rule fixed in the main experiment and examine its sensitivity separately.

2.2 Reinforcement Learning as Optimisation over Future Cost

Conventional dispatching rules often rank actions by their immediate cost. In our problem, however, the effect of moving a job appears later in truck queues, the crane work sequence, and vessel supply. Immediate movement cost alone cannot capture these effects. Reinforcement learning can instead compare actions by the cost that follows over a horizon. Wang et al. apply a PPO-based method to real-time yard-crane scheduling across multiple blocks [8]. Their method optimises work sequencing in the execution layer. We address the preceding assignment layer, where the block and arrival time are selected.

This perspective leads to two modelling choices. First, the networks do not output action probabilities. Each network maps the observed state and one candidate action to its expected operating cost. The policy then selects the candidate with the lowest predicted cost. In this sense, the method follows value approximation, as deep Q-networks approximate action values with neural networks [12], rather than a policy-gradient approach. Second, we use small multilayer perceptrons. Feedforward networks are general function approximators [13], but the input structure also supports this choice. Each input is a fixed-length numeric vector, and candidates are scored independently. The input has no temporal order for a recurrent or attention model to exploit, and the number of labelled decisions is too small to support a much larger model. One set of weights is also shared across all blocks. A small perceptron is therefore an appropriate first model for this setting.

When several parties act, the horizon cost is joint and the effect of one action is difficult to isolate. Wolpert and Tumer construct difference rewards that remain aligned with the global objective while responding to an individual action [9]. Foerster et al. apply the same idea to deep multi-agent learning, where a centralised critic marginalises one agent's action to form a counterfactual baseline [10]. Both obtain the future term from a learned value estimate. We use the same decomposition principle but measure the future term with simulation. Starting from the same state, we rerun the discrete-event simulator after changing one action. The regression target is therefore the monetary cost difference accumulated over a fixed horizon, not a bootstrapped advantage. This choice adds offline computation but keeps the learning signal in the same units as the operating objective.

Kim et al. study learning-based resource control under uncertainty in building demand response. They combine measured environmental records with synthetic parameters for unit-level variation and identify the synthetic component explicitly [14]. Sect. 4.1 follows the same principle by separating literature-based ranges from our design values.

3 Proposed Architecture

3.1 Notation, Actions, and Policies

The decision interval is $\delta = 60\,\mathrm{s}$. The current assignment of job i is $z_i = (b_i, \tau_i)$, where b_i is the block and τ_i is the announced arrival time. A reallocation action is $a_i = (b'_i, \Delta_i) \in \mathcal{A}_i(t)$ and gives $z_i(a_i) = (b'_i, \tau_i + \Delta_i)$. Keeping the original assignment is $a_i^0 = (b_i, 0)$. Block reallocation has $b'_i \neq b_i$, and arrival-time adjustment has $\Delta_i > 0$; a candidate may change both. Outbound jobs are excluded from block reallocation because the retrieved container has a fixed location.

Let n_i be the announcement time, g_i the gate-in time, and r_i an indicator of prior review. The jobs newly eligible at time t are $\mathcal{E}_t = \{i \mid n_i \leq t < n_i + \delta,\ g_i > t,\ r_i = 0\}$. Thus, each job is reviewed once after announcement and before arrival. The proposal policy uses the current block state and job-action features o_i^P. It scores each candidate and selects the one with the lowest expected cost:

$$\hat{a}_i = \arg\min_{a \in \mathcal{A}_i(t)} \hat{c}_\theta^P(o_i^P, a), \qquad p_i = \mathbf{1}[\hat{a}_i \neq a_i^0]. \tag{1}$$

If the policy proposes a change, the destination block or target time slot becomes the accepting party. The acceptance policy uses the proposal message and destination state o_i^A to compare acceptance with rejection:

$$q_i = \mathbf{1}\left[\hat{c}_\psi^A(o_i^A, \hat{a}_i, \text{accept}) \leq \hat{c}_\psi^A(o_i^A, \hat{a}_i, \text{reject})\right]. \tag{2}$$

If the target resource has no remaining capacity, the request is rejected before the network is called. The mutually accepted candidates form $\mathcal{J}_t = \{i \in \mathcal{E}_t \mid p_i q_i = 1\}$.

3.2 Central Commitment Constraints

The central procedure does not create new candidates. It considers only $\mathcal{J}_t$, sorted by the acceptance policy's predicted cost, with lexicographic tie-breaking. Let $x_i \in \{0, 1\}$ indicate commitment, $F_b(t)$ denote the free slots in block b, and $K_k(t)$ denote the remaining quota of time slot k. The procedure enforces

$$\begin{aligned} x_i \leq p_i q_i, \qquad & \sum_{i \in \mathcal{J}_t} x_i \mathbf{1}[b'_i = b] \leq F_b(t), \\ \sum_{i \in \mathcal{J}_t} x_i \mathbf{1}[k(i) = k] \leq K_k(t), \qquad & \sum_{t \in \mathcal{T}} \mathbf{1}[i \in \mathcal{E}_t] \leq 1. \end{aligned} \tag{3}$$

Block capacity is enforced in every experiment. When time-slot quota data are unavailable, we set $K_k(t) = \infty$. Results under this setting represent the potential range of arrival-time adjustment. All committed assignments are applied together at the end of the decision epoch. An earlier commitment therefore cannot change the state observed by another candidate in the same epoch. The structure guarantees two properties: only mutually accepted candidates are committed, and each job is reviewed once.

3.3 Policy Models and Inputs

The functions $\hat{c}^P_\theta$ and $\hat{c}^A_\psi$ in Sect. 3.1 are cost networks. Neither produces action probabilities. Each maps an observed state and one candidate to the expected cost after that candidate is committed. The policy evaluates every candidate with the same network and selects the arg min of the learned cost. Both networks are multilayer perceptrons with weights shared across blocks. The proposal network has a 21-dimensional input, and the acceptance network has a 16-dimensional input. Both use two 64-unit hidden layers with ReLU. They contain 5,633 and 5,313 parameters, respectively, or fewer than 11,000 in total. Inputs include current waiting, inventory, remaining crane work, the public expected arrival time, route difference, and announced volume near the target time. Unrealised arrival and completion times and simultaneous responses from other accepting parties are excluded.

Fig. 2 shows one cost network. A candidate enters as a fixed-length vector $x \in \mathbb{R}^{d_{in}}$. It passes through three linear layers, with ReLU after the first two, and produces one scalar: the expected cost that follows from the candidate. Because the output is one value per candidate, the candidate-list length may change across decision epochs without changing the model. The same weights also apply to every block. The model therefore learns a common rule for evaluating a candidate in the current block state, rather than a separate rule for each block. The policies differ only in input dimension: 21 for proposal and 16 for acceptance.

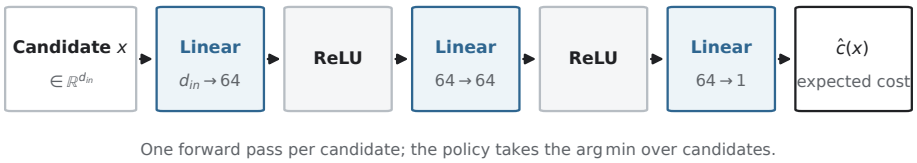

Fig. 2: Structure of one cost network. The input is a fixed-length feature vector for a single candidate and the output is the cost expected to follow from that candidate, so a decision evaluates the same network once per candidate and takes the arg min. The proposal and acceptance policies share this shape and differ only in the input dimension, 21 and 16 respectively.

3.4 Counterfactual Cost Labels

For each sampled decision, we clone the state and random stream just before branching. We then run up to three worlds for three hours:

$$\begin{aligned} \phi_i^F &= \Phi_H(\text{observed}), \qquad \phi_i^P = \Phi_H(\text{proposal reversed}), \\ \phi_i^A &= \Phi_H(\text{acceptance reversed}), \end{aligned} \tag{4}$$

with $H = 10{,}800\,\text{s}$. A decision without an acceptance request produces only the proposal comparison. For each policy, we centre the two actions on their mean, $\bar{\phi}_i^P = (\phi_i^F + \phi_i^P)/2$, and divide by KRW 100,000. This gives $y_{i,F}^P = (\phi_i^F - \bar{\phi}_i^P)/10^5$ and $y_{i,P}^P = (\phi_i^P - \bar{\phi}_i^P)/10^5$. We apply the same centring to (ϕ_i^F, ϕ_i^A) for the acceptance policy. Mean subtraction removes the shared cost without changing the order of the two actions.

3.5 Training Settings and Operational Separation

We sample at most 64 decisions per day. Exploration decreases linearly from 0.50 to 0.05, and decisions are split 80/20 into training and validation sets. We optimise with Adam (3×10^{-4}) and Huber loss ($\beta = 1.0$). A few counterfactual differences are much larger than the rest, so squared error would allow these samples to dominate the fit [17]. Minibatches contain at most 256 rows. The number of updates is $\max(50, \min(600, 4 \times \text{rows}))$, and gradients are clipped at 1.0. Counterfactual branches run in parallel across CPU cores, and each day's labels are replaced after the daily update. During evaluation, weights are frozen, exploration is disabled, and no counterfactual branch is called.

Training both networks for 24 passes took 4.7 CPU hours. Online inference requires one forward pass through a small network for each candidate. This cost is small relative to the accompanying constraint check. The main computational expense is counterfactual label generation, which is performed offline and never enters the operating path.

4 Experimental Environment

4.1 A Literature-Based Synthetic Environment

We evaluate the architecture in a literature-based synthetic environment, not in the operating record of a particular terminal. The studies in Sect. 2 provide the modelling ranges: 20 blocks [7], multiple cranes per block [5], continuous multi-week operation [6], strong time-of-day variation in truck arrivals [11], three vessel classes with different call volumes [7], and a reported range of crane handling times [6]. Within these ranges, we use 21 blocks, two cranes per block, 30 continuous days with state carried across day boundaries, three vessel classes, and a reference crane service time of 180 s. Sect. 4.3 defines the arrival curve. These settings are design choices. The citations show that the values lie within ranges used in prior models; they do not imply calibration to a specific port.

4.2 Terminal and Time Settings

The yard contains 21 blocks and 42 yard cranes, with two cranes per block. Each block has 1,440 slots and starts at 65% occupancy. The gate and quay lie at opposite ends of a one-dimensional layout. The decision interval is 60 s. Operations continue for 30 days, and the middle 28 days are compared. Cranes give priority to service jobs and then to the job with the shortest expected processing time. A day boundary separates cost accounting; it does not reset the terminal. Inventory, queues, crane states, and unfinished jobs carry over. The first and last days connect these continuous boundaries.

4.3 External Truck Demand and the Arrival Process

We construct external-truck demand from ranges reported in prior work. The literature supports similar demand scales and strong time-of-day variation [11]; the specific curve below is our synthetic design. Daily demand is drawn from 3,500, 5,000, 7,500, 12,500, and 15,000 trucks with probabilities 0.30, 0.30, 0.25, 0.10, and 0.05 (Fig. 3a). The first three levels represent smooth, normal, and high demand. The last two represent congested and heavily congested conditions. Within each day, job announcement times follow an arrival density $f(t)$. It combines a uniform base containing 38% of the volume with three Gaussian components containing the remaining 62% (Fig. 3b):

$$f(t) = 0.38\,U(0,24) + 0.62 \sum_{m=1}^{3} w_m \mathcal{N}(t; \mu_m, \sigma_m^2), \tag{5}$$

where (μ_m, σ_m, w_m) equals $(10, 1.5, .317)$, $(15, 2.5, .633)$, and $(21, 1, .05)$, with time in hours. The uniform base keeps nighttime arrivals above zero. The state carried across a day boundary is therefore not created by an artificially empty night. The peak locations, widths, and weights are design values for comparing congestion sensitivity across demand levels, not estimates from a particular terminal. **Training and evaluation use the same curve shape**; only demand level and random draw differ. We therefore do not evaluate transfer to an arrival profile with a different shape.

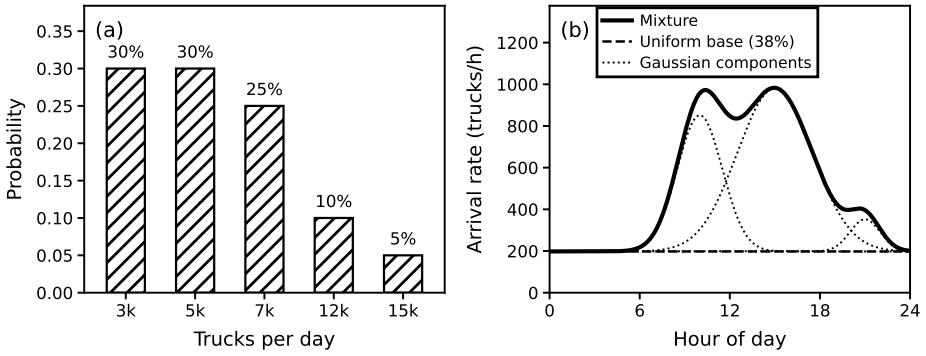

Fig. 3: External truck job demand. (a) The five daily demand levels and the probability with which each is drawn, independently for each day. (b) The time-of-day arrival process at 12,500 trucks per day, with the uniform base and the three Gaussian components drawn separately beneath the mixture they sum to. The literature supports the demand magnitudes and the fact that arrivals are far from uniform over the day, whereas the shape of the mixture is a construction of this study.

4.4 Vessels and Quay Operations

We use three vessel classes. A small vessel has 50,000 GT, 3,000 TEU, and two quay cranes. Its idle cost is KRW 149,500 per hour. A medium vessel has 100,000 GT, 7,500 TEU, and four cranes; its idle cost is KRW 299,000. A large vessel has 150,000 GT, 14,000 TEU, and six cranes; its idle cost is KRW 448,500. For a vessel with capacity C TEU, call volume is $C \cdot u$, where $u \sim U[0.15, 0.30]$. Each day includes two small vessels and one medium vessel; one large vessel is added with probability 0.30. One quay crane processes 27.5 moves/h, within the 24, 30, and 36 moves/h values used in the literature [7]. We sum blocking across assigned quay cranes and divide by their number to obtain idle time per vessel. On the yard side, the reference service time is 180 s per move. Nominal yard capacity is therefore $\mu_{YC} = 21 \times 2 \times 3600/180 = 840$ moves per hour. A move is a container movement, not a truck job. One truck job may require several moves and rehandles, and service time varies by position and job type. We therefore report move and truck units separately.

4.5 Cost Function

The total cost over the interval $[d, d+1)$ is the sum of four terms:

$$\Phi_d = C_{wait,d} + C_{move,d} + C_{rehandle,d} + C_{vessel,d}. \tag{6}$$

Truck dwell costs KRW 40,000 per hour, with the same rate applied a second time beyond one hour. The monetary anchor comes from the safe trucking freight rate, while the time conversion is a design choice [15]. Additional crane movement costs KRW 60,000 per working hour, and rehandling costs KRW 2,000 per event. Both are design values. Vessel idling costs KRW 2.99 per GT-hour, converted from the excess berthing rate of KRW 29.9 per 10 GT-hour [16]. Let h_i be the dwell time of truck i, m the additional crane working time, R the number of rehandles, and T_v the policy-related idle time of vessel v. The terms are

$$C_{wait} = \sum_i 40{,}000\{h_i + \max(0, h_i - 1)\}, \quad C_{move} = 60{,}000\, m, \tag{7}$$

$$C_{rehandle} = 2{,}000\, R, \quad C_{vessel} = \sum_v 2.99\, GT_v T_v. \tag{8}$$

Structural vessel time that the policy cannot change, such as berthing and quarantine, is recorded separately as a diagnostic. All four cost terms are differences between cumulative values measured at the same day boundary.

The 30-day training run reveals two properties of this environment. First, waiting dominates total cost. C_{wait} accounts for 96.4%, compared with 2.2% for rehandling, 1.2% for vessel idling, and 0.1% for additional crane movement. Conclusions based on Φ are therefore driven mainly by the regulated safe-freight-rate coefficient [15]. The two design-based coefficients have little influence. Second, waiting and service levels deteriorate sharply as demand rises. Waiting represents 83.2% of cost at 3,500 trucks per day, 91.7% at 7,500, and 99.1% at 15,000. Across the month, mean truck turn time is 0.89 h and the 90th percentile is 2.01 h. Of 206,904 truck jobs, 5.0% remain unfinished on the day of announcement. More importantly, the daily 90th percentile ranges from 0.79 h on the lightest days to 7.49 h on the heaviest, a 9.5-fold difference. Congested days differ not only in cost level but also in cost composition. The reallocation policy is expected to exploit this change.

5 Numerical Experiments

5.1 Scope of the Results

The results come from a 30-day continuous diagnostic run with random seed 9,900,950, which was not used in training. We compare the middle 28 days and use the first and last days only to connect the continuous boundaries. During evaluation, weights are frozen and exploration is set to zero. All policies share the same truck, vessel, and service random streams on a given day. We record the four cost terms, demand level, transaction types, and unfinished jobs for each day. The source code, raw policy results, and figure scripts are released with the paper.

5.2 Confirmation of Architectural Operation

Implementation checks, commitment constraints, and automated tests confirm four structural properties. Proposal and acceptance use separate models. One-sided consent never produces a commitment. Realised future arrival and completion times are excluded from policy inputs. State also continues across day boundaries. The run logs confirm two additional properties. Frozen-policy evaluation made **zero** counterfactual simulation calls. The identity check also passed at all four sampled points with counterfactual labels: a branch that changed no action reproduced the action and cost of the main run.

5.3 Paired Daily Comparison Across Policies

We run every policy with the same random streams to reduce simulation noise in pairwise differences, following Kleijnen's common-random-number method [18]. Cost difference is defined as proposed minus comparison; a negative value means that the proposed policy costs less. We treat days as independent comparison units and use a two-sided sign test (Table 1). A Wilcoxon signed-rank test, which

Table 1: Policies evaluated on the same 28 days, sorted by cost reduction. The two rightmost columns compare each policy with the proposed policy by day. Policies that adjust arrival time reduce cost, whereas block-only policies remain near zero.

			vs. proposed	
Policy	Action range	Reduction	Days lower	p
Proposed policy	block + time	**+14.72%**	—	—
Proposed, time only	time	+13.44%	—	—
Untrained model	block + time	+7.97%	18/28	0.185
Rule: least-loaded slot	time	+3.41%	—	—
Rule: least-loaded block + slot	block + time	+1.51%	20/28	**0.036**
Rule: least slack	block	+0.53%	20/28	**0.036**
Rule: first-come-first-served	block	+0.17%	22/28	**0.004**
Rule: shortest processing time	block	+0.05%	21/28	**0.013**
Rule: net gain	block	−0.48%	21/28	**0.013**
Proposed, block only	block	−1.31%	—	—
No reallocation	—	0.00%	26/28	**<0.001**

also uses the size of each daily difference, gives the same conclusions. For every rule-based comparison, $p \leq 0.017$.

Over 28 days, total cost was KRW 10.00 bn without reallocation and KRW 8.52 bn with the proposed policy. The reduction was KRW 1.471 bn, or 14.7%. Table 1 reports the reduction for each policy. The four existing rules use block reallocation only, so this comparison does not separate policy quality from action range. Sect. 5.5 provides a matched comparison. The proposed policy was more expensive on only a few days, and those differences were small. The largest reductions all occurred on congested days.

5.4 Demand Level and the Decomposition of Actions

Table 2 decomposes the reduction by demand level. The four days with 12,500 or 15,000 trucks account for 90.5% of the total. This concentration supports developing the architecture toward a policy that *intervenes selectively when congestion is observed* rather than continuously. Restricting the policy to one action type at a time separates the spatial and temporal contributions. Arrival-time adjustment accounts for 91.3% of the total reduction, mostly on the four congested days. Block-only reallocation saves KRW 0.124 bn under smooth and normal demand but raises cost at the highest demand, producing a net increase of KRW 0.131 bn. The policy should therefore adjust its use of spatial and temporal actions by demand level. Time-slot quotas were open in this diagnostic run. The arrival-time results thus describe the *potential range* of time shifting, not an executable schedule. Sensitivity analysis with finite quotas can define the practical range.

Table 2: Cost reduction by demand level (KRW bn, relative to no reallocation). Columns three to five separate the allowed actions. The last column compares the trained and untrained models. Arrival-time adjustment and learning contribute mainly on the four congested days, whereas block reallocation raises cost at the highest demand.

Daily demand	Days	Both actions	Time only	Block only	Due to training
3,500	12	0.047	−0.013	0.057	0.056
5,000	8	0.060	0.029	0.067	0.029
7,500	4	0.032	0.008	0.000	−0.049
12,500	2	0.611	0.573	−0.014	0.144
15,000	2	0.722	0.747	−0.241	0.495
Total	**28**	**1.471**	**1.344**	**−0.131**	**0.675**
share of total		100%	**91.3%**	—	**45.9%**

5.5 Rule-Based Policies of the Same Action Range

To determine whether the difference in Sect. 5.4 comes from the action itself, the rule-based policies must receive the same action range. We therefore transfer the *least-loaded* criterion from the spatial axis to the temporal axis and add two rules. This change introduces no new parameter and uses the same congestion trigger as the other rules. Three findings follow. First, arrival-time adjustment under a rule reduces cost by 3.41% relative to no reallocation ($p = 0.0125$). The temporal action therefore has value by itself. Second, with the full spatio-temporal action range matched, the proposed policy costs less than the rule on 20 of 28 days ($p = 0.036$). Third, for arrival-time adjustment alone, the rule costs less on more days; the proposed policy is lower on only 8 of 28 days. Yet the proposed policy's total reduction is about four times larger. The rule is slightly better under smooth demand, whereas the proposed policy reduces cost 3.8–5.6 times more on congested days. Thus, the learned policy tends to produce large savings on expensive days rather than frequent small wins. Both the number of wins and the total reduction are needed to show this pattern. The rule using both actions also weakens under congestion. This suggests that the congestion sensitivity of block reallocation may be a property of the action, not only of one policy.

5.6 Learning Effect and Dispatching Sensitivity

We first examine model fit. Across 30 training iterations, the proposal network's Huber loss decreases from 0.079 to 0.029 on training decisions and from 0.268 to 0.119 on validation decisions. The acceptance network's loss decreases from 0.171 to 0.027 and from 0.315 to 0.136, respectively. These values are means over the first and last five iterations, with targets in units of KRW 100,000. The decline in validation loss indicates that the fit extends to unseen decisions.

However, final validation loss remains about four times training loss. With about 56 labelled decisions per iteration, this gap supports the choice of a small model. As exploration decreases, the share of decisions with exactly zero counterfactual difference falls from 51.4% to 37.6%. Training runs 4,671 counterfactual worlds in total.

Cost is KRW 10.00 bn without reallocation, KRW 9.20 bn with the untrained model, and KRW 8.52 bn with the trained model. Training therefore adds KRW 0.675 bn, or 45.9% of the total reduction. However, the trained model costs less on only 18 of 28 days ($p = 0.1849$). The mean daily difference is KRW -24.1 m, about 18 times the median of -1.3 m. This gap shows that the learning effect is concentrated rather than evenly distributed. The four congested days account for 94.6% of the learning effect, and the trained model costs less on all four. The corresponding counts at lower demand levels are 9/12, 5/8, and 0/4. For the remaining 24 days, the mean difference is -1.5 m and the bootstrap interval includes zero.

Resampling daily differences within demand strata gives a 95% interval of $[-32, -17]$ m per day for the mean difference. This procedure preserves the demand mix fixed by the design. Without stratification, the interval widens to $[-52, -3]$ m, and 99.2% of resamples remain negative. Both intervals exclude zero, but the stratified interval is partly narrow by construction because each congested stratum contains only two days. The interpretation is therefore limited. Learning produces a large total reduction and is lower on all four congested days, but the day-level sign test is not significant. Establishing the learning effect at a conventional significance level requires more congested days, not simply more days overall.

The ranking of crane-dispatching rules also changes with demand. The shortest processing time rule is 22–25% more expensive than the other rules under smooth and normal demand, but it is the least expensive from 7,500 trucks upward. It also has the lowest overall cost under the designed demand mix. The effect of reallocation should therefore be tested under other dispatching rules.

6 Conclusion

This paper proposed a reinforcement learning architecture that jointly revisits the yard block and arrival time of an announced truck job during operation. The architecture connects distributed appointment negotiation [3] and yard-crane operations [4, 5] in one spatio-temporal assignment problem. Reallocation affects later truck, crane, and vessel operations. Each policy therefore learns from a counterfactual cost difference measured over a fixed horizon, rather than from immediate movement cost. This is how multi-agent counterfactual learning [9, 10] enters the operating decision. Expensive branch simulation is used only during offline training. Online operation requires only small cost networks and constraint checks.

In the literature-based synthetic environment, the architecture carries spatio-temporal reallocation from observation through proposal, acceptance, commit-

ment, and execution. Cost reductions arise mainly from arrival-time adjustments on congested days. This result supports the view that real-time smart-port information can guide assignment actions, not only monitoring [1]. The proposed policy also has lower total cost than a rule with the same action range. For arrival-time adjustment alone, however, the rule is lower on more days, while the proposed policy leads in total because of larger savings on congested days. Both daily win counts and total savings are therefore needed for interpretation.

6.1 Limitations

Seven limitations bound these results. First, the environment is synthetic. Its scale and variation follow reported ranges, but it is not fitted to the operating record of a specific terminal. The demand mix, arrival-curve coefficients, and two of the four cost coefficients are also design values. Second, training and evaluation use the same arrival-curve shape and differ only in demand level and random draw. We therefore do not show transfer to an unseen arrival profile. Third, only four of the 28 days are congested, and most of the reduction occurs on those days. Confirming the learning effect requires more congested days, not simply more days overall. Fourth, we do not isolate the contribution of the independent acceptance policy. We confirm that one-sided consent never produces a commitment, but we do not measure the cost of removing the acceptance veto. Separate proposal and acceptance thus remain a design choice supported by operational checks rather than an ablation. Fifth, time-slot quotas are open, so the arrival-time results give a potential upper range rather than an executable schedule. Sixth, the three-hour counterfactual horizon is a design value. It combines a 30-minute review window, up to 120 minutes of deferral, and a 30-minute arrival-pressure margin. Finally, the main comparison uses one crane dispatching rule. Because rule rankings change with demand, another rule may change the measured size of the reallocation effect.

These limits can be tested one axis at a time. Actual appointment quotas would define the executable range of time adjustment. A wider set of congested conditions would clarify the contribution relative to the untrained model. This study organises observation, proposal, acceptance, commitment, execution, and counterfactual learning into one reproducible structure. As automated equipment and appointment information expand, the architecture can help connect real-time congestion information to explainable resource-assignment decisions.

References

1. Yen, B.T.H., et al.: How smart port design influences port efficiency: a DEA-Tobit approach. Res. Transp. Bus. Manag. **46**, 100862 (2023)
2. Chen, G., Govindan, K., Yang, Z.: Managing truck arrivals with time windows to alleviate gate congestion at container terminals. Int. J. Prod. Econ. **141**(1), 179–188 (2013)
3. Phan, M.-H., Kim, K.H.: Collaborative truck scheduling and appointments for trucking companies and container terminals. Transp. Res. B **86**, 37–50 (2016)

4. Ng, W.C., Mak, K.L.: Yard crane scheduling in port container terminals. Appl. Math. Model. **29**(3), 263–276 (2005)
5. Speer, U., Fischer, K.: Scheduling of different automated yard crane systems at container terminals. Transp. Sci. **51**(1), 305–324 (2017)
6. Petering, M.E.H., Murty, K.G.: Effect of block length and yard crane deployment systems on overall performance at a seaport container transshipment terminal. Comput. Oper. Res. **36**(5), 1711–1725 (2009)
7. Schwientek, A., Lange, A.-K., Jahn, C.: Simulation-based analysis of dispatching methods on seaport container terminals. ARGESIM Report **59**, 357–364 (2020)
8. Wang, W., et al.: Yard crane real-time scheduling among multi-block at terminal: a reinforcement learning based PPO approach. Transp. Res. E **207**, 104635 (2026)
9. Wolpert, D.H., Tumer, K.: Optimal payoff functions for members of collectives. Adv. Complex Syst. **4**(2–3), 265–280 (2001)
10. Foerster, J., et al.: Counterfactual multi-agent policy gradients. In: AAAI, vol. 32 (2018)
11. Kourounioti, I., Polydoropoulou, A.: Application of aggregate container terminal data for the development of time-of-day models predicting truck arrivals. EJTIR **18**(1) (2018)
12. Mnih, V., et al.: Human-level control through deep reinforcement learning. Nature **518**(7540), 529–533 (2015)
13. Hornik, K.: Approximation capabilities of multilayer feedforward networks. Neural Netw. **4**(2), 251–257 (1991)
14. Kim, S., Mowakeaa, R., Kim, S.-J., Lim, H.: Building energy management for demand response using kernel lifelong learning. IEEE Access **8**, 82131–82141 (2020)
15. Ministry of Land, Infrastructure and Transport: Safe trucking freight rates for 2026, Notice 2026-402 (2026). (in Korean)
16. Ministry of Oceans and Fisheries: Port facility charges: berthing rates and calculation basis. (in Korean)
17. Huber, P.J.: Robust estimation of a location parameter. Ann. Math. Statist. **35**(1), 73–101 (1964)
18. Kleijnen, J.P.C.: Analyzing simulation experiments with common random numbers. Manag. Sci. **34**(1), 65–74 (1988)